The Agent Loop Machine: One Semantics, Four Substrates, and Where Capability Actually Lives

Hao Li
Institute of Chemistry, Chinese Academy of Sciences
Beijing, China
leeehow@gmail.com
ORCID 0009-0004-2696-3267
https://github.com/Leehow/seed

August 2026

Abstract

An LLM agent run is usually shipped as one program, but it is three functions: a model μ, an environment ε, and a kernel κ that decides what the model sees next, what the budget is, and when the run is over. We specify κ alone, and then ask what it contributes. The Agent Loop Machine (ALM) is an operational semantics for finite-horizon, single-agent, observation-dependent tool use, with one unusual normative rule: the kernel is *payload-blind*—prompt bytes, tool output and diff hunks are named by opaque references it moves but never reads. What is left is a transition table over small integers and short tokens. We implement it four times on deliberately unlike substrates—CPython, POSIX `/bin/sh`, SQL triggers, and ARM64 assembly—and test whether they are the same machine rather than four similar programs. On 350 recorded conformance cases the four kernels produce byte-identical canonical traces (0 disagreements over 700 runs) and match a separately written, table-driven oracle record for record; 17 of 17 seeded semantic mutations of the reference kernel are caught. Holding μ, ε, the prompt bytes and the budget fixed, per-task success rates across the four substrates differ by at most 0.015 on graded shell work and 0.000 on a live stochastic suite, inside a pre-registered ± 5 pp equivalence band; on 801 Terminal-Bench 2.1 trials under the official verifier the three container-capable substrates differ by at most 0.052, no pair judged different, and that largest difference lives entirely in one of the three repeats. Swapping the model for one from another vendor, with the kernel and everything else held byte identical, moves the same measurement by -0.234—roughly 4.5 times as much as swapping the substrate does. Ablations show what the core is for: removing observation feedback, cross-step state, or the recursion collapses success on witness families built so that no controller lacking that piece can be right on every instance, and bounded repair and retry are worth nothing at fault rate 0 but $+28$ pp at 10%. The four kernels span $1656\times$ in transition throughput and $196\times$ in trusted computing base, yet the slowest is 0.71% of a deliberately conservative 500 ms model call, and 0.07% of the step latency we actually measured. Orchestration substrate is operationally necessary and no longer performance-dominant.

1 Introduction

Agent frameworks are large, and it is not obvious how much of that size is load bearing. The usual way to ask is ablation inside one framework, which answers a question about that framework. We ask a different one: what is the smallest object that still deserves the name *agent loop*, and does the machinery that runs it change what the agent can do?

The question only becomes tractable after a separation. An agent run is commonly written as one program, but it is three functions:

$$a_t \sim \mu(\text{serialize}(s_t)) \tag{1}$$
$$(w_{t+1}, o_t) = \varepsilon(w_t, a_t) \tag{2}$$
$$s_{t+1} = \kappa(s_t, a_t, o_t) \tag{3}$$

μ is the model: stochastic, expensive, and where task competence lives. ε is the environment: a filesystem, processes, a network, and where the work actually happens. κ is the kernel: deterministic, cheap, and the only part that is a *runtime*. Frameworks bundle all three. This paper specifies κ and measures what happens when you keep it and change everything about how it is built.

Our specification, ALM v0.1 [5], adds one rule that does most of the work. A kernel is **payload-blind**: history entries are opaque references minted by the adapter, and a kernel may move them but never read the bytes behind them. Every decision a kernel makes—which step is next, whether the budget is spent, whether an event is stale, whether the run is over—is then a comparison between small integers and short tokens. That is a claim about agent loops, not an implementation trick, and it has a testable consequence: the semantics should fit in a substrate with no string library at all.

We implement ALM four times: a Python reference, a POSIX shell kernel that never forks after start-up, a SQL kernel whose transitions are `WHERE` clauses in database triggers, and an ARM64 assembly kernel using five syscalls. They share one adapter—byte-identical, supplying μ and ε—so that any difference between them is the kernel's.

Contributions.

1. **A specification of κ alone** (§3) with an explicit claim boundary, a wire format narrow enough to parse by byte scan, and a canonical trace that makes two kernels comparable in one hash.
2. **Four conformant kernels** (§4) on substrates that share almost nothing, and a 350-case conformance suite that is mutation-tested so that passing it means something (§6).
3. **Necessity witnesses** (§7): four task families that separate feedback, cross-step state, recursion and stopping, each reported against an information-theoretic ceiling and against live models.
4. **Equivalence rather than absence of significance** (§8): pre-registered ± 5 pp bands, task-clustered bootstrap, and a variance decomposition over a fully crossed model$\times$substrate$\times$task design.
5. **A reliability surface** (§10) and a two-number systems account—kernel-only footprint and trusted computing base—that does not let an assembly kernel pretend libc is free (§11).

What we do not claim. We do not claim ALM is the minimal core for agents in general. It is a candidate minimal core for finite-horizon, single-agent, observation-dependent tool use; multi-agent coordination, human-in-the-loop approval, long-lived memory and real-time control are outside v0.1 by normative text, not by oversight. We do not report a Terminal-Bench leaderboard rank. And we keep the research kernels strictly separate from this repository's production runtime (§12): a 120 KB shell agent with a capability index, packs and a memory system is not evidence about minimality, and calling it that would be the first thing a reader should reject.

2 Related work

Agent loops as state machines. StateFlow models LLM task solving as state-driven workflows [9], and TapeAgents represents an agent's trajectory, state and resumption as one append-only tape [1]. Both establish that agent behaviour is usefully described as state evolution. Neither asks whether the state

machine is portable across computational substrates, nor supplies a conformance criterion by which two implementations could be judged the same machine. That gap—semantics with a test, rather than an architecture—is what §3 and §6 address.

Thin agents. Agentless argues that much agent scaffolding is unnecessary for software repair [10], and mini-SWE-agent shows a small bash-centric loop remains competitive [7]. Quine realises agents as native POSIX processes [4]. These make the case that *less framework* suffices. They do not separate which reductions cost capability from which cost only reliability; our ablations (§7) and fault surface (§10) are aimed exactly at that split.

Agent–computer interfaces. SWE-agent shows that an interface designed for models beats a raw shell on repository repair [11, 3]. We hold the interface fixed at two primitives across every arm precisely because it is known to matter: an experiment that varied the substrate and the tool surface together would measure neither.

Benchmarks and reliability. Terminal-Bench evaluates long-horizon terminal work in real containers with outcome tests [6]; τ-bench measures consistency with pass^k over repeated runs rather than a single attempt [12]. We adopt pass^k and pre-register the Terminal-Bench protocol (§15) without running it here.

Tool making. Voyager and LATM study agents that invent tools over time [8, 2]; ReAct fixes the interleaving of reasoning and action [13]. ALM hard-codes the interleaving in κ so the model cannot rewrite the controller, and says nothing about how tools come to exist.

3 The Agent Loop Machine

3.1 State

$$s_t = (H_t,\ B_t,\ Q_t,\ R_t) \tag{4}$$

$$\begin{aligned}
&H : \text{ordered list of } (\mathit{ref}, \mathit{kind}), \quad \mathit{kind} \in \{\text{MODEL_ARG},\ \text{OBSERVATION},\ \text{REPAIR_NOTE}\}\\
&B = (\mathit{steps},\ \mathit{repairs},\ \mathit{retries},\ \mathit{history_max},\ \mathit{obs_cap})\\
&Q \in \{\text{INIT},\ \text{AWAIT_MODEL},\ \text{AWAIT_TOOL},\ \text{TERMINAL}\}\\
&R = (\mathit{run},\ \mathit{step},\ \mathit{pending},\ \mathit{attempt},\ \mathit{outcome},\ \mathit{reason})
\end{aligned}$$

H is the history, B the budget, Q the phase, and R what a resumption needs.

Wall-clock deadlines are deliberately *not* in B. They are nondeterministic and would make conformance untestable; a deadline is a conforming extension, not part of the core.

3.2 The payload-blind rule

A ref is an opaque token naming bytes the adapter holds. A kernel MUST NOT inspect, decode or concatenate those bytes. Two consequences follow. First, the kernel has no prompt to build and no JSON of its own to render, so its complexity is bounded by the transition table rather than by the model's serialisation format. Second, and more importantly for this paper, agreement between kernels written in four languages is then a fact about the semantics rather than about four copies of the same parser.

The rule also has a design consequence we did not anticipate: because the kernel chooses *which refs* a model request carries but not how they are rendered, every ablation in §7 is a kernel configuration rather than a second prompt. `history_max`$=0$ and `feedback`=off are changes to κ, and the bytes the adapter would have rendered are untouched.

3.3 Transitions

There are 9 transition rules, plus start and the two ignore rules; we state the ones that carry the argument and refer to the specification for the rest. Table 22 lists all twelve with how often the conformance suite fires each.

> **Tool call.** AWAIT_MODEL + *model_response*(OK, TOOL): append the argument ref to H, emit *tool_request*, go to AWAIT_TOOL. The kernel does not validate the tool name: an unknown tool is an environment error, not a kernel concern.
>
> **Observation.** AWAIT_TOOL + *tool_response*(S): append the observation ref *whatever S is*, decrement *steps*, and either emit the next model request or terminate on BUDGET_EXHAUSTED. A failed tool call is information, not an error to swallow.
>
> **Repair and retry.** An ill-formed model answer spends a *repair* and appends a note the model will see; a failed model call spends a *retry* and appends nothing, because nothing happened that the model could learn from. Neither spends a *step*: the budget counts work, not attempts.
>
> **Ignore.** An event is ignored—state unchanged, no effect emitted, one trace record—when its `eid` does not match the outstanding effect, when its type does not fit the phase, or when its type, status or action is not a token the ABI defines. A kernel MUST NOT guess what an unrecognised status meant; in particular it must not treat it as a repair, because a repair asserts that the model answered, and an unparsable record is not evidence that it did.

The ignore rule is what makes a kernel idempotent under at-least-once event delivery, which is what a crash-and-resume adapter actually provides. We measure that property directly in §10.

3.4 The wire, and why it is narrow

Effects and events are flat JSON objects on one line, with no nested values and no escapes; free text never crosses the wire because it lives behind a ref. In the adapter→kernel direction values additionally exclude commas, so that "scan to the next `,` or `}`" is a correct parser. This asymmetry is deliberate: the kernel is the end that must parse without a library, and it is the end whose implementation cost we are trying to measure.

3.5 Canonical trace

Each consumed event produces exactly one trace record:

```
step|phase_in|event|status|action|phase_out|
steps_left|repairs_left|retries_left|refs|flag
```

The SHA-256 over the concatenated records is the *trace hash*. It captures the action sequence, the phase sequence, the budget evolution, which refs the model was shown, and the terminal state, in one comparable string. Two kernels are trace-equivalent on a case iff their hashes are equal. Every matched event produces exactly one effect and every ignored event produces none, so an adapter can run in lock-step without polling—a small property that turns out to matter for writing the assembly kernel.

4 Four kernels

All four implement both profiles (*core*, and the *ext* ablation knobs) and pass the same suite. The shared adapter is 343 logical lines and identical bytes for every kernel, so it cancels in all comparisons.

`py` — CPython. The readable reference. It exists to be obviously correct, not small.

`sh` — POSIX `/bin/sh`. No `jq`, no `awk`, no `sed`: the narrow dialect parses with parameter expansion alone. The first version extracted fields with `x=$(fld ...)`, which forks a subshell per field—about eight forks per event. Assigning to a variable instead removes every fork after start-up. Measured back to back at 100,000 transitions each, the forking variant runs at 170 transitions/s and the assigning one at 2798, a $16.5\times$ speed-up; both pass all 350 conformance cases with no trace disagreement, so this is one kernel measured twice rather than two kernels. The kernel's throughput is entirely a question of whether field extraction forks.

`sql` — SQLite. State is a row, H is a table, and each ALM rule is a guarded `INSERT`/`UPDATE` inside one `AFTER INSERT` trigger; a pre-transition snapshot table keeps guards from reading their own writes. The pump that carries bytes in and out may only insert the raw line, print new `outbox` and `trace` rows, and read `state.phase` once at EOF for its exit status. Delete the pump and the database still contains the whole loop; delete the schema and the pump can do nothing. We state this asymmetry explicitly because the failure mode of a "SQL agent" is a Python `while` loop with a logging database.

`asm` — ARM64 assembly. Five syscalls, no allocator, no string library, no JSON parser. H is a 4096-entry ring: exact for any `history_max` ≤ 4096 and lossy only for an unbounded history longer than that. One source file builds for macOS (Mach-O) and Linux (ELF) and passes the suite on both; the conditional part is five things—syscall numbering, the syscall register, how a symbol's address is spelled, what the sections are called, and the `O_CREAT` bits—and nothing else. It is still absent from the Terminal-Bench arm (§15) because those task images are linux/amd64, and x86-64 is a different instruction set rather than a port.

Writing the second, third and fourth implementation found one real under-specification: the first draft let an unrecognised status fall through to the repair path in two kernels and be ignored in the oracle. The specification now names ignoring as normative and the suite has cases that lock it in.

5 Experimental setup

Within a suite, every arm shares the same system prompt bytes, the same tool surface, the same observation cap, the same step budget and the same fresh working directory; only the named factor varies. The two suites differ from each other by design—the witness families expose a small task-specific tool set, the local tasks expose `shell` and `edit`—and no comparison in this paper crosses them.

Table 1: Models. Decoding parameters are recorded rather than assumed, because endpoints differ in what they accept.

key	vendor	model id	decoding
deepseek-flash	deepseek	`deepseek-v4-flash`	temperature 0, reasoning disabled
deepseek-pro	deepseek	`deepseek-v4-pro`	temperature 0, reasoning disabled
grok-high	xai-via-local-relay	`grok-4.5-high`	temperature 0
grok-low	xai-via-local-relay	`grok-4.5-low`	temperature 0

We attempted a third model from a different vendor and failed twice, in ways worth recording. Two reasoning tiers of that endpoint (`-none` and `-medium`) refused the text protocol outright, halting on turn 1 with "unable to proceed"; a third tier (`-low`) played it correctly in isolation but the endpoint serialised requests at roughly 6.6 s per call and then began returning HTTP 504 under any load, including none. No number in this paper comes from it. The consequence is a real limitation: the model factor below varies *tier within one vendor*, not vendor. The first failure is also a reminder that "the model would not play the protocol" and "the substrate is worse" are easy to confuse if only one model is ever run.

The oracle policy deserves its own note. For the ablation experiments we also run a non-model policy that plays optimally given exactly the entries the kernel let it see, and guesses uniformly when the fact it needs is not there. It is observation-driven rather than step-indexed, so an injected tool failure costs it a step instead of derailing a fixed plan. It computes the information ceiling of an ablation arm with no API and no model variance, which is what separates "this arm cannot work" from "this model could not cope".

6 H1: semantic portability

Table 2: Conformance. 350 recorded cases, 700 kernel runs, 2.4s wall. Cases are generated from a table-driven derivation of the transition rules, written separately from any kernel and by the same author; a kernel passes a case only if its effect stream and its canonical trace match that oracle record for record. Agreement with it is therefore evidence that four substrates implement one specification, not that the specification is right.

kernel	branch	budget	envfault	ext	idempotency	normal	protocol	total
`py`	50/50	50/50	50/50	50/50	50/50	50/50	50/50	**350/350**
`sh`	50/50	50/50	50/50	50/50	50/50	50/50	50/50	**350/350**

All four kernels pass every case, and the cross-kernel trace-hash disagreement count is 0. The agreement is not merely on outcomes: it is on the budget after every transition, on which refs each model request carried, and on which events were ignored.

A second platform. The numbers above come from one host. Running the same 350 cases unchanged inside a Terminal-Bench task container—Linux x86_64, CPython 3.13.12, SQLite 3.40.1, on linux/amd64 emulated over an arm64 host—gives 1050 more runs and the same result (Table 3). The assembly kernel is not in that table because the image is amd64; built from the same source with `cc` in a linux/arm64 container it passes all 350 cases there too. This is worth a paragraph because it caught a real portability bug rather than confirming a belief: the relational kernel had been written against SQLite's ordered `group_concat`, which landed in 3.44, while the task images ship 3.40.1. Unordered `group_concat` would have been a silent wrong answer—the ref list is a sequence, not a set—so the window is now walked with a recursive CTE, available since 3.8.3.

Table 3: The same cases inside a Terminal-Bench task container.

kernel	cases passed
`py`	350/350
`sh`	350/350
`sql`	350/350

Can the suite fail? A suite that four kernels pass is worthless unless it can reject a broken one. We apply 17 single semantic mutations to the reference kernel—each a plausible mistake, not a syntax error—and count how many cases catch each (Table 4). 17 of 17 are caught, the weakest by 10 cases.

The interesting result here is a failure. In the first version the suite was entirely *core* profile, and the mutant that appends observations even when `feedback` is off passed all 300 cases: the suite could not fail on the one knob the paper's central ablation depends on. The *ext* category exists because of that finding, and a rule-coverage table (Table 22) is now generated with the cases—which immediately found a second hole, a rule fired by zero cases.

Table 4: Mutation testing of the conformance suite.

mutation	what it breaks	cases failing	rate
`budget-never-spent`	steps_left is not decremented on a tool response	317	90.6%
`repairs-unbounded`	protocol repair ignores its budget	65	18.6%
`retries-unbounded`	transport retry ignores its budget	27	7.7%
`accepts-stale-eid`	an event answering an old effect is processed	24	6.9%
`terminal-not-absorbing`	events after final are processed again	50	14.3%
`feedback-always-on`	the observation is appended even when feedback is off	17	4.9%
`history-unbounded`	history_max is ignored when selecting refs	45	12.9%
`repair-loses-note`	a protocol repair does not show the model its error	42	12.0%
`retry-appends-note`	a transport retry pretends the model answered	18	5.1%
`transport-is-repair`	a failed model call is charged to the repair budget	10	2.9%
`step-never-advances`	the step counter stays at 1	307	87.7%
`attempt-never-resets`	attempt is not reset at a new step	46	13.1%
`budget-off-by-one`	the run stops one step early	101	28.9%
`halt-status-dropped`	the model's own verdict is not recorded	32	9.1%
`allow-halt-ignored`	the fixed-horizon knob is not honoured	13	3.7%
`refused-halt-spends-repair`	a refused halt is charged to the repair budget	13	3.7%
`unknown-action-repaired`	an unrecognised action is guessed at instead of ignored	26	7.4%

7 H4: what the core is for

Terminal-Bench cannot separate "the agent needed feedback" from "the model happened to write a correct script in one shot". Four witness families can. Each is built so that a controller missing one specific piece of the core cannot be right on every instance, whatever model drives it: FEEDBACK_REQUIRED (the second action depends on a value only the first observation reveals), STATE_REQUIRED (information appears once and is not re-readable), RECOVERY_REQUIRED (the tool fails on purpose and the error names the way out), and UNKNOWN_HORIZON (the number of steps is hidden and overrunning is punished by the world). All four are graded on environment state, never on the halt message, so that arms which cannot halt are still gradeable.

Table 5: Information ceiling per ablation arm: the oracle policy, 50 instances per family, four kernels. No model is involved, so these numbers are properties of the arm.

family	full	fixed horizon	no feedback	no history	one shot
`feedback_required`	1.00	1.00	0.06	0.06	0.20
`recovery_required`	1.00	1.00	0.00	0.00	0.00
`state_required`	1.00	1.00	0.00	0.00	0.00
`unknown_horizon`	1.00	1.00	0.18	0.18	0.00

Of 3440 live ablation runs, 357 hit the adapter's model-call cap, all of them in ablated arms: without the information it needs, a model does not stop, it keeps acting.

Removing feedback, history or the recursion does not degrade these arms; it removes the information on which correctness depends, and the measured rates sit at the guessing ceiling. The variance share attributable to the arm is 0.869, against 0.005 for the family and 0.000 for the substrate.

An honest non-result. `fixed_horizon`—a kernel that refuses to let the model halt—loses nothing on any family. An agent that cannot stop still has a harmless action available in three of the four environments, so removing the stopping rule costs budget and creates overrun risk rather than making a task unsolvable. We report this because the tidy version of this paper would have four collapses instead of three. The core's necessity claim rests on feedback, cross-step state and the recursion; it does not rest on halting.

Table 6: The same matrix with live models.

family	full	fixed horizon	no feedback	no history	one shot
deepseek-flash, 50 instances per family					
`feedback_required`	1.00	1.00	0.00	0.00	0.00
`recovery_required`	1.00	1.00	0.00	0.00	0.00
`state_required`	1.00	1.00	0.00	0.00	0.00
`unknown_horizon`	1.00	1.00	0.00	0.00	0.00
deepseek-pro, 15 instances per family					
`feedback_required`	1.00	–	0.20	0.00	0.00
`recovery_required`	1.00	–	0.00	0.00	0.00
`state_required`	1.00	–	0.00	0.00	0.00
`unknown_horizon`	1.00	–	0.00	0.00	0.00
grok-low, 50 instances per family					
`feedback_required`	1.00	1.00	0.06	0.00	0.00
`recovery_required`	1.00	1.00	0.00	0.00	0.00
`state_required`	1.00	1.00	0.00	0.00	0.00
`unknown_horizon`	1.00	1.00	0.00	0.00	0.00

Table 7: Arm comparisons against `full`, paired by instance, 10,000-resample task-clustered bootstrap, ± 5 pp equivalence band.

arm	rate	diff vs full	95% CI	verdict	p (Holm)
fixed_horizon	1.000	+0.000	[+0.000, +0.000]	equivalent	1
no_feedback	0.060	+0.940	[+0.905, +0.970]	different	1.53e-56
no_history	0.060	+0.940	[+0.905, +0.970]	different	1.53e-56
one_shot	0.050	+0.950	[+0.920, +0.980]	different	5.1e-57

8 H2 and H3: substrate against outcome

Table 8: Live witness suite, `full` arm, 3 repeats, 2400 runs. pass^k is the unbiased "succeeds k times out of k" estimator.

kernel	runs	rate	pass^1	pass^3
`asm`	600	1.000	1.000	1.000
`py`	600	1.000	1.000	1.000
`sh`	600	1.000	1.000	1.000
`sql`	600	1.000	1.000	1.000

The claim being tested is equivalence, not absence of significance. "We found no significant difference" is compatible with an underpowered experiment; a 95% CI for the paired difference lying inside a band declared in advance is not. Every substrate pair falls inside ± 5 pp on both suites.

Sample size was set by that rule rather than by taste, and the first attempt failed it. With 12 tasks at 3 repeats the paired bootstrap CI was wider than the band and three of the six pairs returned *inconclusive*—a statement about the experiment, not about substrates. Doubling the suite to 24 tasks and raising repeats to at least three per arm narrowed the interval enough for a verdict. We report this because "inconclusive" is the outcome a pre-registered band is for, and because the fix is more data rather than a wider band.

8.1 The model factor

The local suite cannot carry this comparison. Its model levels changed protocol at the same time they changed vendor, so model and prompt are confounded in it, and on the current model it is saturated

Table 9: Pairwise substrate equivalence on the live witness suite.

A	B	rate A	rate B	diff	95% CI	verdict	p (Holm)
asm	py	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1
asm	sh	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1
asm	sql	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1
py	sh	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1
py	sql	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1
sh	sql	1.000	1.000	+0.000	[+0.000, +0.000]	equivalent	1

anyway—all four substrates resolve every one of its 1920 runs. A ceiling separates nothing.

So the model factor is measured where there is headroom: one substrate, one protocol, the same 89 Terminal-Bench tasks, `qwen3.8-max` against `grok-4.5-low`, with everything except the model byte identical.

Changing the vendor moved the resolved rate by -0.234. That headline mixes two things: the slower model timed out on 29 of the 89 tasks, and a timeout is a wall-clock failure rather than a competence failure. Stratifying separates them—on the tasks where it never ran out of time the gap is still -0.164, with a 95% interval that excludes zero—and the faster model resolves the tasks the slower one timed out on at close to its own average rate, so those are not simply the hard tasks being reshuffled into one column.

Set that beside the substrate comparison on the same arm, where no pair differed by more than 0.052 and none was ever judged different: **changing the model moved the outcome by roughly 4.5 times as much as changing the substrate did.** That is the paper's claim in one comparison, and it is measured rather than argued.

Two honest limits on it. The contrast is one vendor pair, not a survey; and because the two models differ in speed as well as in ability, part of the gap is a property of this benchmark's wall clock rather than of the models. Both are visible in Table 13 rather than folded away.

9 Terminal-Bench 2.1

The suites so far are ours. This one is not: 89 container tasks with outcome tests written by other people, run under Harbor with the official verifier. The protocol was fixed in advance (§15): dataset checkout `7131e43`, 80 agent steps, the same two primitives, the same prompt bytes, three repeats, and the three container-capable substrates run as three concurrent jobs so that each sees the same slice of wall clock and the same endpoint conditions.

Three models appear in this section and the reason is worth stating plainly. The first arm ran on `deepseek-v4-flash` and was retired after 1560 trials because it was costing real money on a metered endpoint; the replacement is served by a local relay at no per-token charge. That is a budget decision, not a scientific one, and it costs comparability—so the retired arm is reported rather than overwritten. The third arm is a single substrate on `qwen3.8-max` from a different vendor, and exists only to put a number on the model factor (§8.1).

What was excluded, and why it matters. Two kinds of failure end up in the same result file and must not be pooled. An agent that runs out of wall clock has failed, and is scored 0. An environment that will not build has not told us anything about the agent, and is dropped. 10 trials were dropped and 23 were scored 0 under that rule (Table 16); 1 of the exclusions land in repeat 0, nearly all of them `docker compose` failures while the host pulled 60 GB of task images for the first time. That is a reason to report repeats separately rather than to average them away, and it is the kind of accounting that decides whether a substrate looks worse because it is worse or because its first pass ran during an image storm.

Table 10: Local end-to-end tasks: 24 graded shell tasks with real setup and real verifiers, across 4 substrates and 4 models at 3–7 repeats per arm—the two arms that ran first ran more—for 1920 runs in total. Grading is on environment state by a verifier script, not on the agent's report.

task	deepseek-flash	deepseek-pro	grok-high	grok-low
`c-build`	1.00	1.00	1.00	1.00
`cli-args`	1.00	1.00	1.00	1.00
`count-lines`	1.00	1.00	1.00	1.00
`csv-filter`	1.00	1.00	1.00	1.00
`dedupe-csv`	1.00	0.00	1.00	1.00
`dep-error`	0.93	0.96	1.00	1.00
`dir-tree`	1.00	1.00	1.00	1.00
`env-default`	1.00	1.00	1.00	1.00
`exact-edit`	0.00	0.50	1.00	1.00
`fix-python`	1.00	1.00	1.00	1.00
`fix-shebang`	1.00	1.00	1.00	1.00
`grep-count`	1.00	1.00	1.00	1.00
`hello-file`	1.00	1.00	1.00	1.00
`json-extract`	1.00	1.00	1.00	1.00
`make-executable`	1.00	1.00	1.00	1.00
`patch-config`	0.14	1.00	1.00	1.00
`pipeline-report`	1.00	1.00	1.00	1.00
`rename-ext`	1.00	1.00	1.00	1.00
`sort-uniq`	1.00	1.00	1.00	1.00
`sqlite-query`	1.00	1.00	1.00	1.00
`sum-json`	1.00	1.00	1.00	1.00
`tar-roundtrip`	0.00	1.00	1.00	1.00
`two-file-refactor`	1.00	0.93	1.00	1.00
`word-count`	1.00	1.00	1.00	1.00
all	**0.878**	**0.933**	**1.000**	**1.000**

The result, stated as it came out. Of the three pairs, 0 are *equivalent*, 3 are *inconclusive*, and 0 is *different*. The largest point difference is 0.052, and its interval reaches just past the band, so the pre-registered rule does not let us certify equivalence. McNemar finds no significant difference on any pair (smallest Holm-adjusted p is 0.36).

That largest difference deserves its own sentence, because it would be easy to present it either as nothing or as a finding. It is `py` against `sh`, and Table 18 shows where it comes from: the first two repeats put the two substrates within 0.017 of each other and disagree about which is ahead, and the whole of the pooled difference comes from the third. The 28 tasks the two disagree on split 18–10 in `sh`'s favour, which a sign test does not separate from chance ($p = 0.19$). We read that as a single repeat's luck at a base rate near 0.5, where per-task variance is at its maximum—but we report the pooled number, the per-repeat decomposition and the non-significant test, and leave the reader to disagree.

So the honest summary is: no evidence of a substrate effect on any pair, and not enough resolution to certify its absence at ± 5 pp on this benchmark at this sample size.

That the intervals are wide is not a defect of the design so much as a property of the operating point. The substrates disagree on 37 of 89 tasks, always by one or two repeats out of three and in both directions, with nothing concentrated on any substrate—the signature of a model that is unstable on hard tasks rather than of a runtime that is worse. Only 21 tasks are solved by every substrate on every repeat, and only 62 are solved by anything at all: with a two-primitive kernel, no exam-style delivery prompting, and this model, most of Terminal-Bench 2.1 is out of reach, and a low base rate with high per-task variance is exactly the regime in which ± 5 pp is expensive to certify.

Table 11: The same runs by substrate. pass^5 is computed per (task, model) cell, so a substrate that got lucky once does not read as consistent.

substrate	deepseek-flash	deepseek-pro	grok-high	grok-low	all	pass^5
`asm`	0.881	0.952	1.000	1.000	0.942	0.902
`py`	0.881	0.940	1.000	1.000	0.938	0.881
`sh`	0.875	0.917	1.000	1.000	0.927	0.866
`sql`	0.875	0.923	1.000	1.000	0.929	0.866

Table 12: Pairwise substrate equivalence on graded shell work, paired by task, pooled over models.

A	B	rate A	rate B	diff	95% CI	verdict	p (Holm)
`asm`	`py`	0.942	0.938	+0.004	[-0.004, +0.015]	equivalent	1
`asm`	`sh`	0.942	0.927	+0.015	[+0.002, +0.033]	equivalent	1
`asm`	`sql`	0.942	0.929	+0.013	[+0.000, +0.031]	equivalent	1
`py`	`sh`	0.938	0.927	+0.010	[+0.000, +0.023]	equivalent	1
`py`	`sql`	0.938	0.929	+0.008	[+0.000, +0.019]	equivalent	1
`sh`	`sql`	0.927	0.929	-0.002	[-0.006, +0.000]	equivalent	1

What moved the absolute rate. Between the first arm and the second the resolved rate went from 0.171 to 0.465. Every step of that came from the adapter or the model; the kernels, the ABI and the two primitives were untouched, and the substrate verdicts are unaffected because all three substrates always share one adapter, byte for byte.

Most of the change has a single identifiable cause, and it is a result about prompt design rather than a defect report. Protocol `a` instructed the model to emit an action and nothing else, and the model ran with reasoning disabled. Between them, those two choices leave the model no channel in which to notice that its last command failed—so it re-issues it. In one run inspected live, 25 actions contained 9 distinct commands, one of them repeated verbatim ten times. That observation is not reproducible from the artifact and is not offered as evidence: the canonical trace is payload-blind by construction, so it records that an action was issued and under which reference, never the command text. The two claims that follow are recomputed from the traces and are. Across the arm the median run spent 80 of its 80 steps and halted voluntarily 12% of the time. Protocol `b` changes one thing, requiring a short thought before the action, and the median falls to 16 steps with voluntary halts at 36%. Both figures are recomputed from the canonical traces by `alm/bench/trace_stats.py` and published as `trace-stats.jsonl`, one row per trial.[1]

The interesting part is the size of the correction relative to what it did not fix. Removing the loop is worth +0.028 on the same 89 tasks: real, and an order of magnitude less than the gap to a mature harness. The loop was burning steps that were not going to solve those tasks anyway.

Two ways to sharpen the substrate verdicts exist and both are honest: more repeats, which narrows the interval without changing the design, or a stronger model, which raises the base rate. We report the pre-registered three repeats rather than adding data until a verdict appears.

[1]Two integration details cost us runs and are recorded here for anyone wiring a reasoning model into a tool loop rather than as findings. First, `max_tokens` covers reasoning tokens: a model can spend the whole allowance thinking and return empty content, which an adapter should not score as a protocol error. Second, a container cannot reach a relay on the host's loopback address; rewriting the model URL for that case must not also rewrite it for an endpoint on the public internet. The second cost a whole pilot, which died after one step per trial with three transport errors and a clean abort—the kernel doing exactly what §3 says while the adapter fed it a broken URL.

Table 13: Changing the model, holding the kernel, the protocol, the tools, the budget and the task set fixed. The second row removes the tasks where the slower model ran out of wall clock, because a timeout is not the same kind of failure as a wrong answer.

stratum	tasks	qwen3.8-max	grok-4.5-low	diff	95% CI	CI excludes 0
all tasks	89	0.258	0.493	-0.234	[-0.339, -0.129]	yes
slower model did not time out	60	0.383	0.547	-0.164	[-0.294, -0.036]	yes

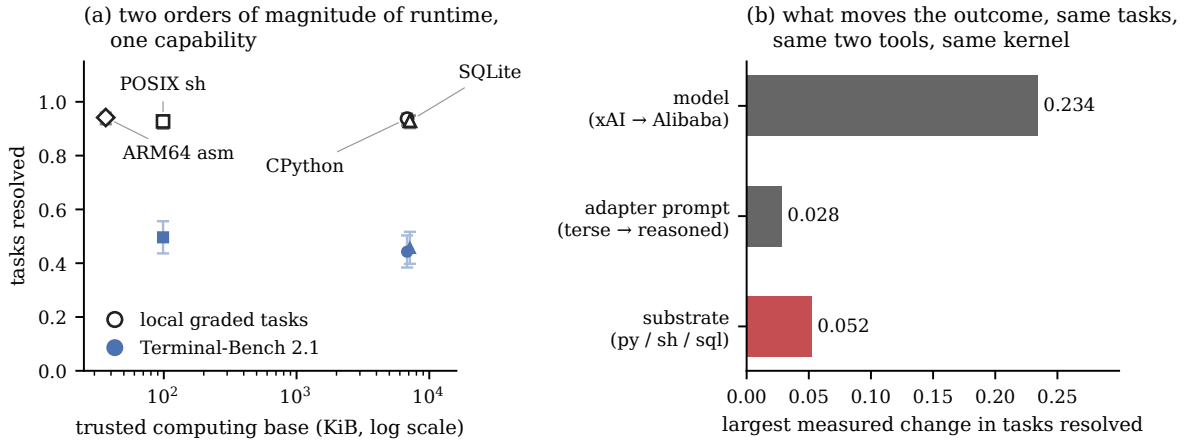

Figure 1: The paper in one picture. *(a)* Trusted computing base against tasks resolved, for the local graded suite (all four substrates) and for Terminal-Bench 2.1 (the three that run in a Linux container, on the pre-registered main arm, `grok-low` under protocol `br`). The horizontal axis spans two orders of magnitude; the vertical axis does not move with it. CPython and SQLite differ by 5% in trusted computing base (6802 against 7141 KiB), so their markers coincide; the leaders identify them. Bars are Wilson intervals on the pooled trials, which are wider than the task-paired intervals used for the equivalence verdicts and are shown here only to keep the picture honest about sample size. *(b)* The same outcome, measured against the three things one can change while keeping the two primitives and the specification fixed. Swapping the substrate is much the smallest of the three, and the two that do move it are both on the model's side of the boundary.

10 Reliability

Capability is not the only thing a runtime is for. We corrupt four channels at the same rate λ: the model returns unparsable output, the model call fails in transport, the tool fails or times out, and every event is redelivered with probability λ. The policy is held fixed, so what moves is the runtime's problem.

Three results. First, bounded repair and retry buy *nothing* in a clean world (1.000 against 1.000 at $\lambda = 0$) and $+28$ pp at $\lambda = 0.10$ (1.000 against 0.720). Second, dropping them makes runs *cheaper* in model calls, because a run that aborts early spends fewer: the wrong way to read a cost table, and a good illustration of why efficiency numbers without an outcome column are misleading. Third, across the sweep 14288 events were redelivered and 0 produced a duplicate side effect, on every substrate—absorbing redelivery is a property of the specification, not of one implementation.

At $\lambda = 0.5$ the full arm falls to 0.520. The core's error handling is bounded on purpose; it converts an unreliable world into a slower one up to a point, and then reports `protocol_exhausted` rather than pretending.

Table 14: The arms. Protocols belong to the adapter, never to the kernel: `a` forbids commentary, `b` requires a thought before the action, `br` is `b` with an output budget wide enough for a model that reasons before it answers.

model	protocol	tasks	substrates	repeats	trials	resolved	role
`deepseek-v4-flash`	`a (terse)`	89	3	5	1263	**0.171**	retired on cost
`deepseek-v4-flash`	`b (reasoned)`	89	3	1	266	**0.199**	prompt contrast
`grok-4.5-low`	`br`	89	3	3	791	**0.465**	substrate comparison
`qwen3.8-max`	`br`	89	1	1	89	**0.258**	model factor

Table 15: 2461 trials: 89 tasks $\times$ 3 substrates $\times$ 1 repeats. Rates are over scored trials.

substrate	rep 0	rep 1	rep 2	all	resolved	pass^3
`py`	0.461	0.483	0.384	**0.443**	116/262	0.286
`sh`	0.477	0.471	0.539	**0.496**	131/264	0.360
`sql`	0.494	0.404	0.471	**0.457**	121/265	0.345

11 Systems

"Cold" is the first invocation of a run, whose pages, dynamic-linker caches and (for the relational kernel) schema are not yet warm; "warm p50" is the median of the remaining 199. The gap is a few milliseconds and matters only for one-shot use.

Two sizes are reported for every kernel and both belong in any honest account. Quoting kernel-only bytes alone is how an assembly kernel becomes "a 9 KB agent" while a JSON parser and libc sit underneath it; quoting only the TCB hides the thing the paper is about. Lines of code appear last and with a warning: one line of SQL, shell and assembly are not the same unit. The substrate-independent complexity numbers are 9 transition rules and 13 mutable control-state fields, fixed by the specification and implemented in full by each kernel.

Figure 1 puts the two sizes and the outcomes on one plot. The four kernels span 1656$\times$ in transition throughput, and that spread is almost invisible end to end: the slowest kernel costs 3561 μs per transition. Against a deliberately conservative 500 ms model call that is 0.71%; against the median step latency actually observed on Terminal-Bench (4.8 s across 2408 scored trials) it is 0.07%, and measured orchestration time is 0.13% of wall clock across every live run in this paper, against 98.8% spent waiting for the model. This is the quantitative form of the paper's central negative result. Orchestration substrate is operationally necessary—something has to hold the budget, the phase and the idempotency—and it is no longer where the time goes.

12 Relation to a production runtime

The kernels above are research instruments. The same repository contains a production agent, `seed.sh`: a single POSIX shell file with a capability index, prompt packs, skills, a memory lifecycle, delegation and run evidence. It is roughly 10^5 bytes, and it is *not* evidence for anything in §§6–11. We keep the two apart because conflating them is the easiest way to make a minimality claim that is not true: a file with a memory system is not a minimal core, whatever language it is in.

For context, and with the caveats stated where the numbers are—including that these runs predate this study and their job directories were not kept, so unlike everything else here the figures cannot be recomputed from the artifact—on the official Terminal-Bench 2.1 suite (89 tasks, Harbor, official verifier, k=1, one shared model) `seed.sh` resolved 45/89 (mean 0.506), against 66/89 (0.742) for mini-SWE-agent and 62/89 (0.697) for Terminus-2 under the same model and harness. The gap is concentrated in `AgentTimeoutError` (32 versus 12 and 11), not in a collapse of correctness among work that reached the verifier, and the install surfaces are not equal—mini-SWE-agent's Harbor adapter is given a Python toolchain, while the Seed container is given curl and CA certificates. That comparison is a

Table 16: Disposition of the trials that did not simply pass or fail.

disposition	outcome	trials
excluded	`VerifierTimeoutError`	8
excluded	`no_reward`	2
scored 0	`AgentTimeoutError`	22
scored 0	`NonZeroAgentExitCodeError`	1

Table 17: Pairwise equivalence on Terminal-Bench, paired by task, pooled over repeats, 10,000-resample cluster bootstrap, pre-registered ± 5 pp band.

A	B	rate A	rate B	diff	95% CI	verdict	p (Holm)
`py`	`sh`	0.440	0.493	-0.052	[-0.105, -0.004]	inconclusive	0.355
`py`	`sql`	0.440	0.455	-0.015	[-0.073, +0.045]	inconclusive	1
`sh`	`sql`	0.493	0.455	+0.037	[-0.011, +0.086]	inconclusive	0.359

systems report about a production agent at k=1; it is not a leaderboard rank, and it is not a measurement of ALM.

13 What follows for someone building one

Nothing in this paper tells anyone to write an agent loop in SQL. What the measurements do support is a short list of things to spend effort on, and one to stop spending it on.

Effort spent on the loop's implementation buys little. Across 801 Terminal-Bench trials, 2400 live witness runs and 1920 graded local runs, no substrate pair was ever judged different, and the largest difference we could produce lived in a single repeat. Meanwhile one change of model moved the same benchmark by -0.234. If a team is deciding where to put a week of engineering, the loop is the wrong place; the prompt, the model and the tool surface are where the outcome moves.

A loop worth keeping is small enough to specify and test. The kernel here is 9 transition rules over 13 state fields, and that is small enough to write a conformance suite for. It was worth it: 350 recorded cases caught 17 of 17 seeded mutations, including several—an unbounded repair budget, a stale event accepted, a history window ignored—that would otherwise surface as rare misbehaviour under load rather than as a failing test. Any agent loop can be given the same treatment; the payload-blind rule is what makes the cases cheap to record.

Size the error handling to the failure rate you actually have. Bounded repair and retry are worth nothing in a clean world—1.000 against 1.000 at zero fault rate—and 28 points at a 10% fault rate. Teams running against a flaky endpoint should not read the first number, and teams running against a good one should not pay for machinery sized by the second.

Two integration facts that cost us runs. An output budget must cover reasoning tokens, or a reasoning model returns an empty completion that looks like a protocol violation and is not. And a model that cannot see its own last failure will re-issue it: forbidding commentary while also disabling reasoning removed the only channel the model had, and the median run went to the step limit (80 of 80 steps) instead of finishing in 15.

Table 18: The same substrate differences, one repeat at a time. A difference that is real should persist across repeats; one that lives in a single repeat is the model being unstable on hard tasks.

repeat	py − sh	py − sql	sh − sql
0	-0.017	-0.034	-0.017
1	+0.011	+0.078	+0.067
2	-0.156	-0.088	+0.068

Table 19: Fault surface. `no_repair`, `no_retry` and `no_recovery` remove the corresponding budgets from the kernel.

arm	λ	runs	success	model calls/run	redelivered/run	duplicate effects
`full`	0.0	800	1.000	3.52	0.00	0
`full`	0.01	800	1.000	3.64	0.06	0
`full`	0.05	800	1.000	4.28	0.37	0
`full`	0.1	800	1.000	5.08	0.98	0
`full`	0.3	800	0.930	7.72	3.86	0
`full`	0.5	800	0.520	7.96	5.89	0
`no_recovery`	0.0	800	1.000	3.52	0.00	0
`no_recovery`	0.1	800	0.720	3.12	0.56	0
`no_recovery`	0.3	800	0.305	2.48	1.05	0
`no_repair`	0.0	800	1.000	3.52	0.00	0
`no_repair`	0.1	800	0.850	3.63	0.64	0
`no_repair`	0.3	800	0.505	3.60	1.64	0
`no_retry`	0.0	800	1.000	3.52	0.00	0
`no_retry`	0.1	800	0.840	4.21	0.78	0
`no_retry`	0.3	800	0.515	4.30	2.04	0

14 Limitations

- The end-to-end task evidence is 24 local tasks and four witness families, not a 89-task container benchmark. The Terminal-Bench matrix is pre-registered and unrun (§15); until it is run, H2 is established on smaller and easier work than the community standard.
- The model factor is one vendor pair, not a survey, and the two models differ in speed as well as in ability, so part of the gap in Table 13 belongs to this benchmark's wall clock. Both limits are shown in that table rather than folded away.
- Three models were used across the study and they are not interchangeable: the substrate verdicts come from one of them, and the arm that established them first was retired mid-study on cost. Everything is reported per arm, but a single arm carried on one model throughout would have been cleaner.
- The largest single lesson here is negative and was expensive: 1301 trials were paid for—1263 of them reaching a verdict—before a trace was read carefully enough to notice the agent was re-issuing the same failed command. Cheap qualitative inspection should precede expensive quantitative runs, and in this study it did not.
- The witness families are designed to separate arms. They are evidence about the core, not an estimate of how often real tasks need feedback.
- The assembly kernel runs on macOS and Linux but only on arm64, and its history ring holds 4096 entries.

Table 20: Footprint and performance. *TCB* is the interpreter or binary, every library it links, and—for the Python-backed kernels—exactly the standard-library modules they import, not the whole install. Libraries in the macOS dyld shared cache have no file on disk and count as zero, so `sh` and `asm` are understated by libSystem.

kernel	substrate	src KiB	gzip KiB	LOC	TCB KiB	cold ms	warm p50 ms	transitions/s	peak RSS MiB
`py`	cpython 3	9.1	2.6	212	6801.6	16.5	17.1	177,139	21.8
`sh`	POSIX /bin/sh	6.8	2.2	180	98.7	6.4	7.2	2,975	2.5
`sql`	SQLite relational	15.1	3.3	239	7141.3	37.2	32.9	280	104.8
`asm`	ARM64 assembly	39.6	7.2	1330	36.5	6.0	2.8	463,755	1.7

Table 21: Where the wall clock and the tokens go. T_{orch} is everything that is neither the model call nor the tool: prompt rendering, the ABI round trip, and the kernel.

suite	model	runs	T_{model}	T_{tool}	T_{orch}	orch ms/step	prompt tok/run	completion tok/run
local tasks	deepseek-flash	672	92.2%	7.5%	0.37%	3	6853	253
local tasks	deepseek-pro	672	81.1%	18.6%	0.37%	6	1532	95
local tasks	grok-high	288	99.1%	0.7%	0.15%	7	4945	775
local tasks	grok-low	288	98.9%	0.9%	0.17%	7	4196	501
witness ablations	deepseek-flash	1600	99.5%	0.0%	0.47%	3	2932	39
witness ablations	deepseek-pro	240	99.5%	0.0%	0.52%	6	1142	16
witness ablations	grok-low	1600	99.9%	0.0%	0.06%	4	5041	2817

- The fault injector corrupts four channels with independent Bernoulli draws. Real failures are correlated and bursty.
- Everything here is single-agent and finite-horizon by construction.

15 Pre-registration, and what is still not run

The Terminal-Bench protocol was fixed before any of it was run: frozen dataset commit and per-task manifest, frozen decoding and budget parameters, a ± 5 pp equivalence band with the bootstrap and correction named in advance, and a 30-task subset selected by $\mathrm{sha256}(\mathrm{salt}\|\mathrm{task})$ under proportional allocation by difficulty and written to disk before any result existed. §9 is that protocol executed on the primary model.

One deviation, declared: the pre-registration interleaves substrates inside each task block; we instead ran the three substrates as three concurrent jobs, which exposes them to the same wall clock and the same endpoint state more tightly than block interleaving would.

Two deviations are on the record and neither was decided by looking at an outcome. The first arm's amendment promised five repeats and was stopped after four repeats and 233 of the fifth repeat's 267 trials (87%), on cost. One repeat of the second arm was discarded and re-run after the host filled its disk mid-arm: that repeat recorded seventeen infrastructure faults against the neighbouring repeat's four, and the discarded directories are kept beside the replacements so the claim can be checked. A single-substrate arm on a third model was discarded on the same grounds, twenty-three faults against one.

Still unrun: the same-model external baselines, which bear on the production runtime of §12 rather than on any hypothesis here. The assembly kernel is absent from the Terminal-Bench arm because those images are linux/amd64.

16 Conclusion

Separating κ from μ and ε, and forbidding κ to read payload bytes, leaves an agent loop small enough to write in assembly and in SQL triggers, and precise enough that four such implementations agree on

every one of 350 recorded cases, trace record by trace record. Holding the model and the environment fixed, which of them runs the loop does not measurably change what the agent accomplishes, while it changes the trusted computing base by two orders of magnitude. What the core does buy is visible elsewhere: feedback, cross-step state and the recursion are load-bearing for capability, and bounded repair, retry and idempotency are load-bearing for reliability—worth nothing in a clean world and 28 points at a 10% fault rate. The useful question about an agent runtime is therefore not how small it can be made, but which of its parts are paying for capability, which for reliability, and which for neither. On the evidence here the substrate is paying for neither, and the parts that are paying can be named.

Artifacts

Specification, four kernels, conformance suite, witness families, statistics and the generated result tables: `https://github.com/Leehow/seed` under `alm/`. Every table and every macro in this paper is generated from a results file by `alm/paper_tables.py`, which refuses to build if the text uses a macro no results file defines. Three numbers in the running text are not covered by that: the §12 comparison against mini-SWE-agent and Terminus-2, whose runs predate this study and whose job directories were not kept; and the duplicate-command observation in §9, which the payload-blind trace cannot record. Each is marked where it appears. The per-trial trace statistics behind the step-count claims are published as `alm/bench/trace-stats.jsonl`; the raw job trees they are derived from are several gigabytes and are not.

A Rule coverage of the conformance suite

Table 22: Every transition in the specification and how often the suite fires it. Generated with the cases.

rule	specification	trace records	cases
`start`	s4.1 start	350	350
`tool`	s4.2 model calls a tool	1195	317
`halt`	s4.3 model halts	204	204
`halt_disabled`	s8 halt refused (ext)	23	13
`repair`	s4.5 protocol repair	62	45
`abort_protocol`	s4.5 repairs exhausted	35	35
`retry`	s4.5 transport retry	23	18
`abort_transport`	s4.5 retries exhausted	13	13
`observe`	s4.4 observation, run continues	1097	307
`abort_budget`	s4.4 budget exhausted	98	98
`stale_event`	s4.6 event ignored	384	50
`after_terminal`	s4.6 event after terminal	102	50

References

[1] Dzmitry Bahdanau, Nicolas Gontier, Gabriel Huang, Ehsan Kamalloo, et al. TapeAgents: a holistic framework for agent development and optimization. *arXiv preprint arXiv:2412.08445*, 2024.

[2] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[4] Hao Ke. Quine: Realizing LLM agents as native POSIX processes. *arXiv preprint arXiv:2603.18030*, 2026.

[5] Hao Li. ALM v0.1: an operational semantics for tool-using agent loops. `https://github.com/Leehow/seed/tree/main/alm/spec`, 2026.

[6] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

[7] SWE-agent team. mini-SWE-agent. `https://github.com/SWE-agent/mini-SWE-agent`, 2025.

[8] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[9] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. *arXiv preprint arXiv:2403.11322*, 2024.

[10] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[12] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.